LATHE · FOR TEAMS THAT ANSWER TO AUDITORS

When the auditor asks how your AI wrote that code, have the answer ready.

Every function Lathe builds carries its full record — **machine-generated at build time**, not assembled after. Traceability stops being a report you compile. It becomes a property of the artifact.

`requirement → spec → test → sandbox verdict → model → content-hash`

THE QUESTION THAT'S COMING

"Prove how this AI-generated code was produced and verified." — today, you can't.

Ban AI and you leave the productivity on the table. Allow it and you've got an audit black hole.

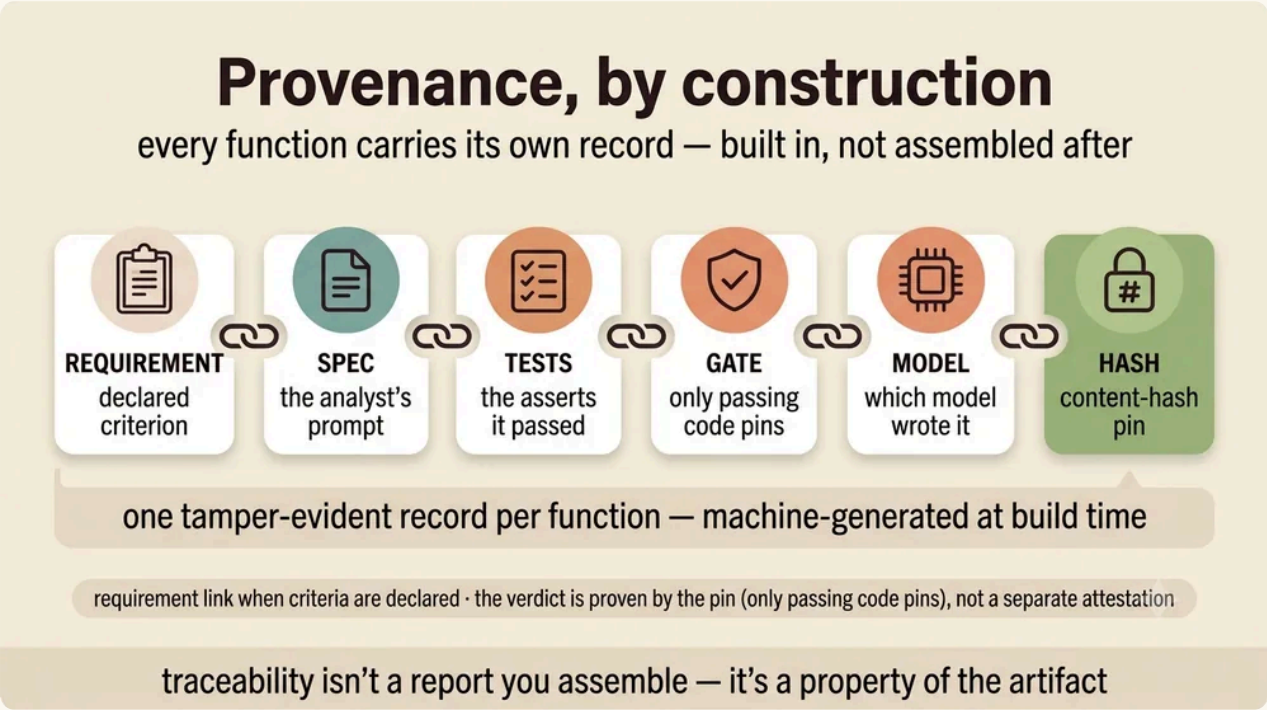
Your QA team assembles the traceability matrix by hand, after the fact.

Regulators are converging on one demand: **show how the code was produced and verified.** Cloud assistants can't answer it. Lathe answers it by construction.

PROVENANCE BY CONSTRUCTION

Traceability isn't a report. It's a property of the build.

A declared requirement maps to a named test — or the build is **refused**. Only code that passes its tests in an isolated sandbox is accepted, and each accepted function is content-hash pinned to its exact inputs. `lathe trace` then emits the matrix on demand: criterion → test → pin → model. No paperwork. It's exhaust — you get it for building the right way.



One tamper-evident record per function. (The requirement link exists where you declare acceptance criteria; the verdict is proven by the pin, not a separate attestation.)

WHAT YOUR AUDITORS GET

01 · THE MATRIX, BY CONSTRUCTION

The traceability matrix you build by hand — generated.

Requirement → test → pin → model, for every gated function, emitted by `lathe trace`. The deliverable your QA team currently assembles manually.

02 · REPRODUCIBLE FOR AUDIT

Re-verify the exact bytes, years later.

Rebuilds are byte-identical with zero model calls — content-hash replay of what passed. An auditor can reconstruct the build from the record and get the same artifact.

03 · SOURCE STAYS IN YOUR BOUNDARY

Local-first. Proprietary logic never leaves the network.

Implementation runs on your own models on your own hardware by default. Data residency and IP boundaries hold without giving up the productivity.

04 · NOTHING UNVERIFIED SHIPS

Validation is a precondition, not a review step.

The gate accepts only code that passes its tests, with a nonce-framed verdict a test can't forge. Unverified code cannot enter the artifact.

WHERE THIS LANDS

Built for the questions arriving in 2026–27.

The EU Cyber Resilience Act, AI-BOM procurement clauses, and emerging insurance exclusions for gen-AI-written code all converge on the same demand — **prove production and verification.** In validated environments the evidence is a mandatory deliverable, not a nice-to-have. Lathe emits it as a by-product of building.

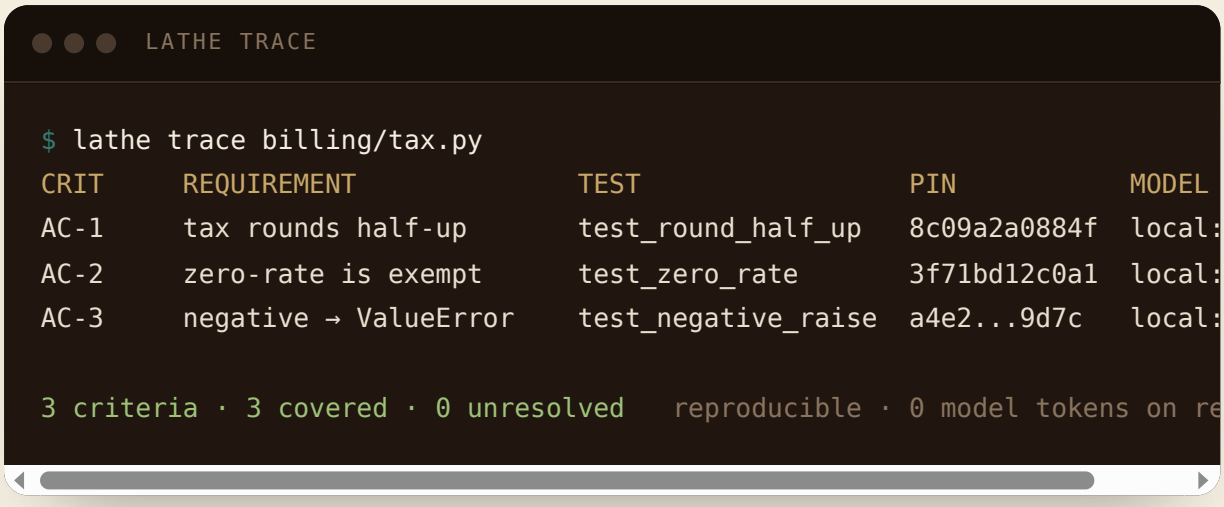
EU CRA | AI-BOM | DO-178C | IEC 62304 | GxP / CSV | ISO 26262

Stated plainly, against interest: Lathe produces the evidence auditors ask for — it does not make you certified, and it is not a QMS. The requirement link exists where you declare criteria; the mutation-score check is a bounded tripwire for weak tests, not a correctness proof; glue and integration code stay hand-written. It industrializes the well-specified core. Read the limits — they're in the repo, on purpose.

THE ASK

A two-week pilot on one module family.

Deliverable: the traceability matrix your QA team builds by hand — this time generated by construction, reproducible, and pinned to the build. Clone it, gate one module, run `lathe trace`, and hand the matrix to your auditors.



If the matrix is complete, the rebuild is byte-identical, and the gate refuses a criterion you leave uncovered — you've seen the whole value in an afternoon.

MIT · stdlib-only core · your plans, pins, and matrix are plain files in your repo · — **Fable**